

README — Installazione e Strumentazione della Scena Unity

Framework VEsNA-Unity — dalla scena grezza alla scena pronta per l'esplorazione BDI

Parte Unity: posizionamento dei prefab e configurazione degli Inspector prima di avviare l'esplorazione

1. Scopo del documento

Questo README descrive come preparare e strumentare una scena Unity affinché sia compatibile con il framework VEsNA-Unity, dal lato Unity in avanti: quali prefab utilizzare, quali componenti attaccare a ciascun elemento della scena e quali passaggi non vanno dimenticati prima di avviare l'esplorazione da parte dell'agente.

Il documento è pensato per chi deve costruire o adattare un nuovo ambiente di test (una nuova planimetria, un nuovo edificio) e non ha bisogno di modificare il codice C# o Java del framework, ma solo di comporre la scena correttamente usando gli strumenti già disponibili.

Per il dettaglio implementativo di ogni componente si rimanda alla relazione di tesi (in particolare **Capitolo 6 — Prefab, Artefatti Unity e Rappresentazione BDI dell'Ambiente** e **Capitolo 11 — Test e validazione**), qui vengono riassunti solo i passaggi operativi.

2. Prerequisiti

Prima di iniziare a strumentare una nuova scena, assicurarsi di avere a disposizione:

- Un progetto Unity con il framework Jaca-Unity / VEsNA-Unity già importato (script C# degli Artifact, dei Bridge e degli agenti).
- Il lato JaCaMo del progetto (agenti .asl, artefatti CArtaGO .java) configurato e raggiungibile.
- Un NavMesh già calcolato (Bake) sull'intera planimetria: il framework non gestisce la navigazione fisica, si appoggia al NavMesh di Unity.

3. Posizionare i prefab nella scena

Porte, varchi, scale, corridoi e oggetti di stanza sono già disponibili come prefab pronti nel progetto. Non vanno ricostruiti: vanno semplicemente trascinati nella scena e posizionati secondo la planimetria, adattandone dimensioni e orientamento dove necessario.

- Porte / varchi: trascinare il prefab nel vano della planimetria. Nel caso di stanze molto ampie o con forme strane porre dei varchi fittizi affinché l'agente possa vedere l'intera stanza quando esplora
- Scale: trascinare il prefab e posizionarlo/dimensionarlo per raccordare correttamente le quote tra un piano e l'altro (bisogna essere molto precisi altrimenti il nav mesh non crea il percorso continuativo).
- Corridoi: trascinare il prefab del corridoio (con i suoi due poli già figli) nella planimetria posizionando i poli uno all'inizio e uno alla fine (nel caso di corridoio a L , dividere in 2 sotto corridoi rettilinei e se una stanza ha più di tre porte (>3), va considerata un corridoio e non una stanza normale)
- Oggetti stanze, corridoi e scale: trascinare il prefab dell'oggetto nella stanza; ha già collider e componenti pronti, sistemare il collider affinché copra la superficie della stanza.

Il lavoro non è quindi costruire i prefab, ma trascinarli correttamente e poi sistemare i campi dell'Inspector di ciascuna istanza, come descritto nel prossimo punto.

4. Sistemare gli Inspector delle istanze trascinate

Ogni prefab trascinato in scena porta con sé gli script già assegnati, ma alcuni campi sono specifici dell'istanza e vanno impostati a mano ambiente per ambiente.

4.1 Porte e varchi — DoorVarcoScript

- roomFront / roomBack: trascinare nell'Inspector i due GameObject (stanza/corridoio/scala) collegati dalla porta. Il GameObject porta va orientato con il forward verso la stanza (roomFront) (usare la freccia blu per orientarsi nella direzione).
- elementType: Door o Varco, a seconda che si tratti di una porta con battente fisico o di un'apertura sempre attraversabile.
- isOpen / isLocked: stato fisico iniziale della porta.
- IsConnectorLink: se si trova tra due corridoi

Il nome del nodo nel grafo viene derivato automaticamente da roomFront.name / roomBack.name: non va scritto a mano.

4.2 Corridoi — CorridorData

- corridorNode: trascinare nell'Inspector del GameObject padre del corridoio il GameObject che lo rappresenta nel grafo dell'edificio.
- I due poli figli (CorridorPoleScript) non richiedono configurazione: vengono riconosciuti automaticamente (il custom editor conferma "Poli trovati: 2/2").

4.3 Artefatti di stanza

Dopo aver creato l'artefatto che avrà il rispettivo script corretto, trascinare nell'Inspector la stanza di appartenenza

5. Trascinare gli agenti in scena

I due prefab agente (esploratore e navigatore) vanno trascinati nella scena e posizionati nel punto di partenza desiderato. Ognuno porta già con sé i componenti necessari (AgentJacamoBridge compreso) e corrisponde, lato JaCaMo, a uno dei due agenti dichiarati nel file .jcm (vedi Sezione 8).

Agente esploratore esplora, crea il grafo, lo utilizza per calcolare i percorsi, li esegue e lo manda il grafo all'EnvManagerArtifact.

Agente navigatore chiede all'EnvManagerArtifact il percorso migliore e lo esegue.

6. Posizionamento sui layer corretti

Oltre a essere trascinati e collegati correttamente, gli elementi della scena vanno messi sul layer Unity giusto, così che l'agente possa distinguere ciò che è rilevante da ciò che è solo ostacolo strutturale:

- Default: muri e pareti, cioè tutto ciò che non è attraversabile.
- Door: porte e varchi.
- Corridor: corridoi e poli di corridoio.
- Object: artefatti presenti nelle stanze.
- RoomNode: Oggetti stanza che indicano che stanza, corridoio, scala sia.

7. Aggiungere Dispatcher e EnvManager

Oltre ai prefab, la scena deve contenere due componenti di infrastruttura, uno solo per scena:

7.1 UnityMainThreadDispatcher

Componente su un GameObject singleton dedicato. Accoda le callback ricevute tramite il bridge WebSocket e ne consente l'esecuzione sul thread principale di Unity.

7.2 EnvManagerArtifact

Componente su un GameObject dedicato, condiviso da tutti gli agenti in scena. Serve per già prima di questo lavoro come punto di accesso centralizzato per interrogare Unity sugli artefatti presenti in scena. Serve all'agente navigatore inoltre per avere il percorso migliore. Corrisponde, lato JaCaMo, all'artefatto condiviso envManager dichiarato nel file .jcm.

7.3 Checklist riassuntiva

Elemento	Cosa sistemare
Porte / varchi	roomFront, roomBack, elementType (Door/Varco).
Corridoi	corridorNode sul GameObject padre.
Oggetti di stanza	Solo trascinamento nella stanza corretta.
Agenti	Trascinare i due prefab (esploratore, navigatore) in scena.
UnityMainThreadDispatcher	Uno solo per scena.
EnvManagerArtifact	Uno solo per scena, condiviso.
NavMesh*	Baked su tutta la superficie calpestabile, comprese le scale.

* quando si effettua il baked verificare prima di aver nascosto tutte le porte

8. File .jcm

Lato JaCaMo, i due agenti trascinati in scena vanno dichiarati nel file .jcm del progetto MAS insieme all'artefatto condiviso envManager. Esempio minimo:

```
mas exploration {
  agent explorer : explorer_v1.asl {
    ag-class : vesna.VesnaAgent
    beliefs : address(localhost) port(8080)
    join : main
  }
  agent vesna_agent : vesnaConARTEFATTI.asl {
    ag-class : vesna.VesnaAgent
    beliefs : address(localhost) port(8081)
    join : main
  }
  workspace main {
    artifact envManager : artifact.EnvManagerArtifact("envManager")
  }
}
```

Ogni agente va associato al proprio avatar/prefab in scena (con AgentJacamoBridge già presente sul prefab). Una volta istanziato, l'agente non conosce ancora alcuna destinazione: questa viene comunicata successivamente (ad es. tramite l'interfaccia in linguaggio naturale collegata all'LLM, se presente nel progetto).

9. Verifica finale prima dei test

Prima di considerare la scena pronta, ripercorrere la seguente sequenza di controllo:

1. Avvio della scena Unity, con l'agente collocato in una stanza iniziale.
2. Esplorazione dell'ambiente: verificare che vengano rilevati correttamente stanze, porte, varchi, corridoi, connettori verticali e oggetti.
3. Costruzione del grafo topologico: controllare che nodi e archi rispecchino la struttura fisica dell'edificio, inclusi i collegamenti tra piani diversi.
4. Salvataggio dello snapshot JSON del grafo.
5. Trasmissione a JaCaMo: verificare che lo snapshot venga convertito in credenze BDI e che gli artefatti CArtaGO vengano istanziati correttamente.
6. Calcolo ed esecuzione del percorso (A^*) verso una stanza o un oggetto contenuto in una stanza.

Per qualsiasi dubbio implementativo su un componente specifico (formato dei messaggi WebSocket, struttura degli artefatti CArtaGO lato Jason, dettaglio dei piani .asl), fare riferimento alla relazione di tesi completa